

Day 1

JDK : <https://www.oracle.com/java/technologies/downloads/#jdk21-windows>

IDE : <https://www.jetbrains.com/idea/download/?section=windows>

Boiler-plate || Default Code

```
public class Demo1 {  
    public static void main(String[] args)  
    {  
  
    }  
}
```

Class- Collection of methods and variables. It is concept of OOPs(DTL).

Method - A method is a block of code which performs certain operations and returns output(DTL).

Entry Point

The main method is the entry point for executing a Java application. When you run a Java program, The JVM or java compiler looks for a public static void main(String args[]) method in that class, and the signature of the main method must be in a specified format for the JVM to recognise it as its entry point.

If we update the method's signature, the program will throw the error **NoSuchMethodError:main** and terminate.

Day 2

Just like we have some rules that we follow to speak English(the grammar), we have some rules to follow while writing a Java program. The set of these rules is called Syntax.

Comments

Single-line comment:

- Use : `//`
- Example : `// This is single line comment`

Multi-line comment:

- Use : `/* */`
- Example : `/* This
 is
 multiline comment */`

Variables

A variable is a container that holds data. This value can be changed during the execution of the program.

Before use, you need to declare and define it.

1. Variable Declaration:

```
int age;  
String name;  
// int and string are the data types
```

2. Variable Initialization:

```
age = 69;  
name = "The boys";  
// int and string are the data types
```

3. Combined Declaration and Initialization:

```
int age = 69;  
String name = "The boys";
```

4. Final Variables (Constants):

```
final int a = 7;[DTL]
```

Role of + operator between String & numbers

- String + String = String - Concatenation
- String + int = String - Concatenation
- int + int = int - Arithmetic Addition

Day 3

Identifiers- Identifiers are used to uniquely identify the variables. Identifier is a name given to a variable, class, method, package, or other program elements.

Rules for Identifiers in Java:

1. **Start** : Must start with an alphabet or _ or \$ NOT with a digit.
2. **End** : Can end with an alphabet or _ or \$ or numeric digit.
3. **No Reserved Words** : You cannot use Java's reserved words (also known as keywords) as identifiers.
4. **No Special Symbols** : Identifiers cannot contain special symbols like @, #, %, etc. except for underscores (_) and dollar signs (\$).
5. **No Space** : Spaces are not allowed.
6. **Length** – No Limit

Java is CASE SENSITIVE : Shery and shery is different for java

camelCase	Used to name methods and variable eg- main(), last name
PascalCase	Used to name classes, interfaces(yet to come)
snake_case	can be used in place of camel case(not recommend)
kebab-case	Unsupported in java

Keyword and word : Keywords are reserved(built-in) words which has specific meanings and cannot be used as identifiers.e.g.public, class, static, if, else, while etc.

DAY 4

Literal or Constant:

Any constant value which can be assigned to the variable.

DATA TYPES

Data types are used to classify and define the type of data that a variable can hold.

There are 2 types of Data Types:

1. Primitive Data types : pre-defined, fixed size.
2. Non-Primitive Data types : Customize and no fixed size.

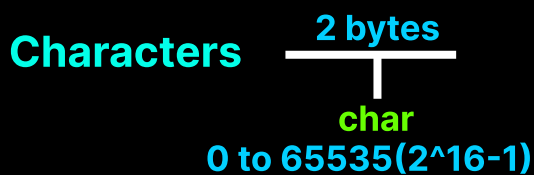
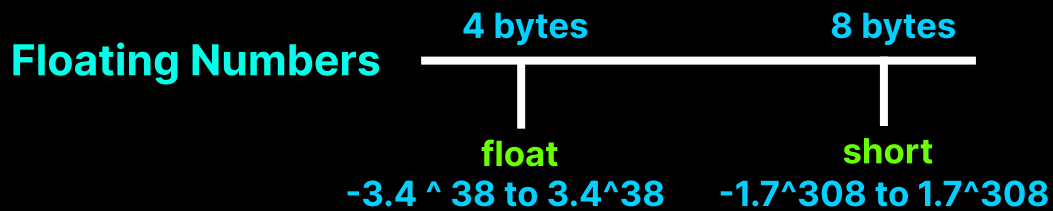
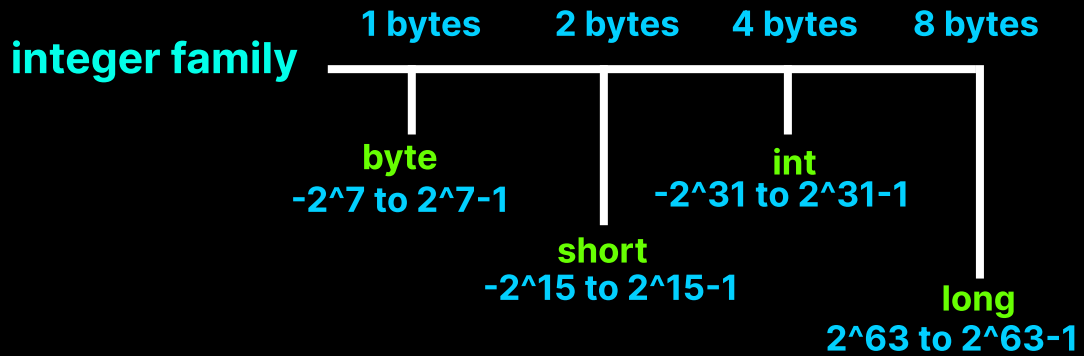
Default Values

Data Types	Value
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0d
char	'\u0000'
boolean	false

the compiler never assigns a default value to an uninitialized local variable(DTL).

Data types

Primitives



Non-Primitives

User defined classes eg- string

+ operator between two char values

- It performs addition between their Unicode code points.
- For example :



```
char a = 'a';  
char b = 'b';  
System.out.println(a+b); //97+98=195
```

Day 5

Scanner

To take input from users we use Scanner class.

Scanner class is a built-in class in the java.util package(DTL). Before using the Scanner class you have to import the Scanner class using the import statement as shown below:

To use the Scanner class, you need to create an object of it, and then you can use that object to interact with the input data.

Example -



```
import java.util.Scanner;  
Scanner sc = newScanner(System.in); //object  
int n = sc.nextInt();
```

The nextInt() method parses the token from the input and returns the integer value.

Use methods to read respective data

nextByte(), nextShort(), nextInt(), nextLong(), nextFloat(), nextDouble(), nextBoolean()

Reading String Data -

nextLine() - Reads the whole line

next() - Reads the first word

Reading Char Data : `next().charAt(0)`

Problem with `nextLine()` method:

If we try to read String after reading in an Integer, Double or Float etc.

Java does not give us a chance to input anything for the name variable.

When the method `input.nextLine()` is called Scanner object will wait for us, to hit enter and the enter key is a character ("`\n`").

Example -



```
Scanner scanner = new Scanner(System.in);

System.out.print("Enter an integer: ");
int age = scanner.nextInt();

System.out.print("Enter a string: ");
String name = scanner.nextLine();

System.out.println(name + " age is = " + age);
```

Console :

Enter an integer: 69

Enter a string: age is = 69

1. We first prompt the user to enter an integer age using `nextInt()`.
2. After reading the integer, we immediately hit enter and enter is also a character represented by "`\n`" – 69
`\n`

3. The int value 69 is assigned in age but not the \n still left in the memory or buffer.
4. In next line when we Call nextLine() to consume the name it first check in buffer is there any thing as we have \n in buffer it take \n (for nextLine() method \n is the stopping point it will consider we stop giving input and return) and skip the line.

Solutions -



```
Scanner scanner = new Scanner(System.in);

System.out.print("Enter an integer: ");
int age = scanner.nextInt();

// Consume the newline character left in the
input buffer
scanner.nextLine();

System.out.print("Enter a string: ");
String name = scanner.nextLine();

System.out.println(name + " age is = " + age);
```

1. After taking an integer input we Call nextLine() to consume the name character left in the input buffer. In next line when we Call nextLine() to consume the name character left in the input buffer.
2. Then, we prompt the user to enter a string using nextLine().

Escape Sequence(\)

\n (next Line), \b (backspace), \t (tab), \" (double quote), \' (single quote), and \\ (backslash).

Day 6

Operators

Operators can be easily defined as characters that represent an operation. These symbols perform different operations on several variables and values.

Example : $5 + 6 = 11$.

Here, 5 and 6 are the **operands**, and + is called the **operator**.

Categories of Operators

Unary operators : perform an action with a single operand.

Binary operators : perform an action with a two operand.

Types of Operator

1. Arithmetic Operator :

- **Binary Operators :** **+**, **-**, *****, **/** (int/int will always yield int) , **%**
(Return remainder after dividing two numbers & with int (works perfectly) but with float (produces ambiguity)). Special powers of **/** & **%** by powers of 10 **/** : to reduce the number by 1 digit **%** : to get last digit(s) of number.

Unary Operators :

- **Increment Operator (++)** : Increase the value by 1.
- **Decrement Operator (--)** : Decrease the value by 1. +
- **-, - and ! (DTL)** is also a unary operator (-5, 5 (is same as +5)).

RULES for Increment and Decrement :

- Cannot applied to constant

Example : `int c = ++10; // compile-time error`

- Nesting of both operators is not allowed

Example : `int a = 10; int b = ++(++a); // compile-time error [++11]`

- They are not operated over final variables

Example : `final int a = 10; int b = ++a; // compile-time error`

- Increment and Decrement Operators can not be applied to booleans.

Example : `boolean a = false; a++; // compile-time error`

Quiz On Increment And Decrement Operators →

2. Relational Operators :

Used to check the relations between two operands. They return a boolean value (true or false) by comparing the two operands. Greater Than (>), Less Than (<), <=, >=, ==, !=

- Equal To (==)

Checks if two operands are equal.

- Not Equal To (!=)

Checks if two operands are not equal.

- Greater Than or Equal To (>=)

Checks if one operand is either greater than or equal to the other.

- Less Than or Equal To (<=)

Checks if one operand is either less than or equal to the other.

3. Logical operators :

Combine multiple conditional statements. There are three types of logical operators in Java: **AND(&&)**, **OR (||)** and **NOT(!)** operators.

Logical AND Operator(&&)

Returns true when both conditions under evaluation are true, otherwise it returns false.

e.g : `if(a>b && a);`

Logical OR Operator(||)

Returns true if any one of the given conditions is true, otherwise it returns false. It returns false if and only if both conditions under evaluation are false.

e.g : `if(a>b || a<c) System.out.println("Max : " + a);`

Logical Not Operator(!)

It accepts a single value as an input and returns the inverse of the same.

This is a unary operator unlike the AND and OR operators.

e.g : `if(!(a<c) System.out.println("Max : " + a);`

4. ShortHand operators

The assignment operator can be combined with other operators to build a shorter version of the statement. **`+=`, `-=`, `*=`, `/=`, `%=`**

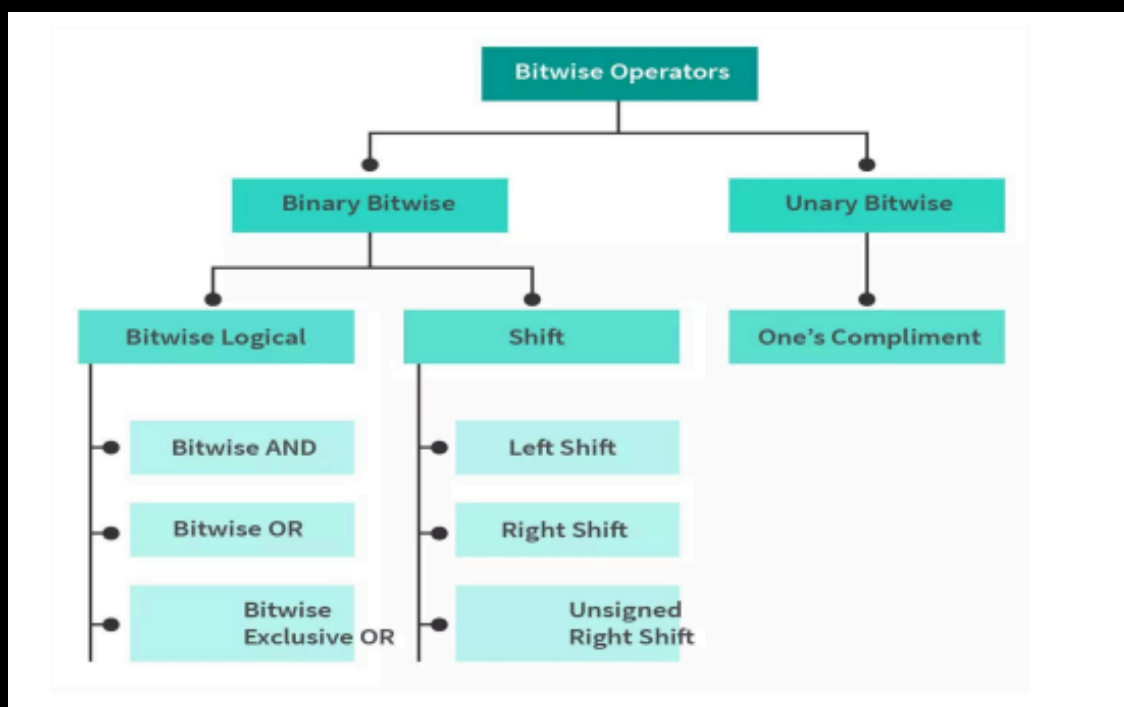
Example : `a = a+5`, we can write `a += 5`.

do not use `==` & `!=` [(`=`) followed by a unary plus (+)]

Bitwise Operators

Bitwise operators work on a binary equivalent of decimal numbers

- First, the operands are converted to their binary representation
- Next, the operator is applied to each binary number and the result is calculated
- Finally, the result is converted back to its decimal representation



Bitwise Logical

1. Bitwise AND(&) : If both the bits are 1, the solution has 1 in that bit position else 0.

Its Truth Table:

- $1 \& 0 \Rightarrow$ gives 0
- $0 \& 1 \Rightarrow$ gives 0
- $0 \& 0 \Rightarrow$ gives 0
- $1 \& 1 \Rightarrow$ gives 1

Example : Perform Bitwise AND Operation of 6 and 8 (6 & 8)

6 = 0110 (In Binary), 8 = 1000 (In Binary) 0110 & 1000 -> 0000 = 0 (In decimal).

2. Bitwise OR(|): if any bits is 1 then it will give 1

Its Truth Table:

- $1 | 0 \Rightarrow$ gives 1
- $0 | 1 \Rightarrow$ gives 1
- $0 | 0 \Rightarrow$ gives 0
- $1 | 1 \Rightarrow$ gives 1

Example : Perform Bitwise AND Operation of 6 and 8 (6 & 8)

6 = 0110 (In Binary), 8 = 1000 (In Binary) 0110 | 1000 -> 1110 = 14 (In decimal)

2. Bitwise XOR(^)

If the bits are opposite, the solution has a 1 in that bit position and if they are matched, a 0 is returned.

Its Truth Table:

- $1 \wedge 0 \Rightarrow$ gives 1
- $0 \wedge 1 \Rightarrow$ gives 1
- $0 \wedge 0 \Rightarrow$ gives 0
- $1 \wedge 1 \Rightarrow$ gives 0

Example : Perform Bitwise AND Operation of 6 and 8 (6 & 8)

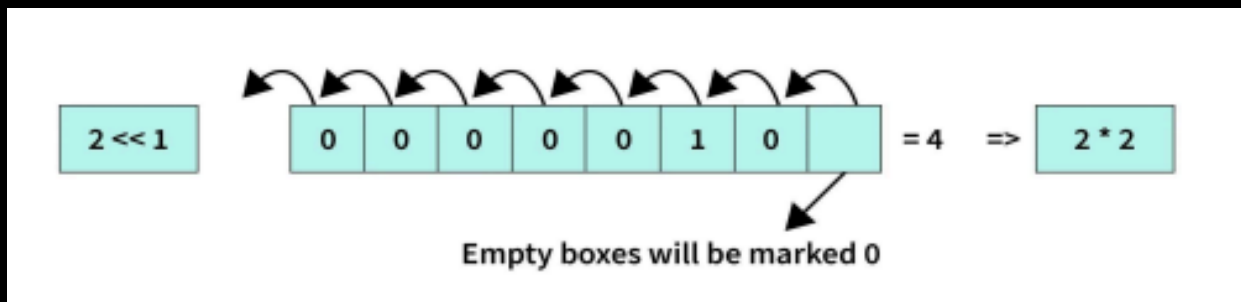
**6 = 0110 (In Binary), 8 = 1000 (In Binary)
0110 ^ 1000 -> 1110 = 14 (In decimal)**

Bit Shift (>>, <<,>>>)

Shifts each digit in a number's binary representation left(<<) or right(>>) by as many spaces as specified by the second operand.

There are three types of shift:

1. Left shift: << : $2 \ll 1$



- $2 \ll 1 \Rightarrow 0010 \ll 1 \Rightarrow 0100 \Rightarrow 4$ (in decimal).
- Add as no. of 0's on the right side of no. as no. of digit you need to shift.

2. Signed right shift: >> : $8 \gg 2$

- $8 \gg 2 \Rightarrow 1000 \gg 2 \Rightarrow 0010 \Rightarrow 2$ (in decimal).

4. Bitwise Complement (~)

- Bitwise Not or Complement operator invert each input bit. This inverted cycle is called the 1's complement of a bit series.
- All samples of 0 become 1, and all samples of 1 become 0 Example :
 ~ 6 (means 1's complement of 6) $\sim 6 \Rightarrow -7$ [Trick $\Rightarrow (-n+1)$]

Day 7

Package

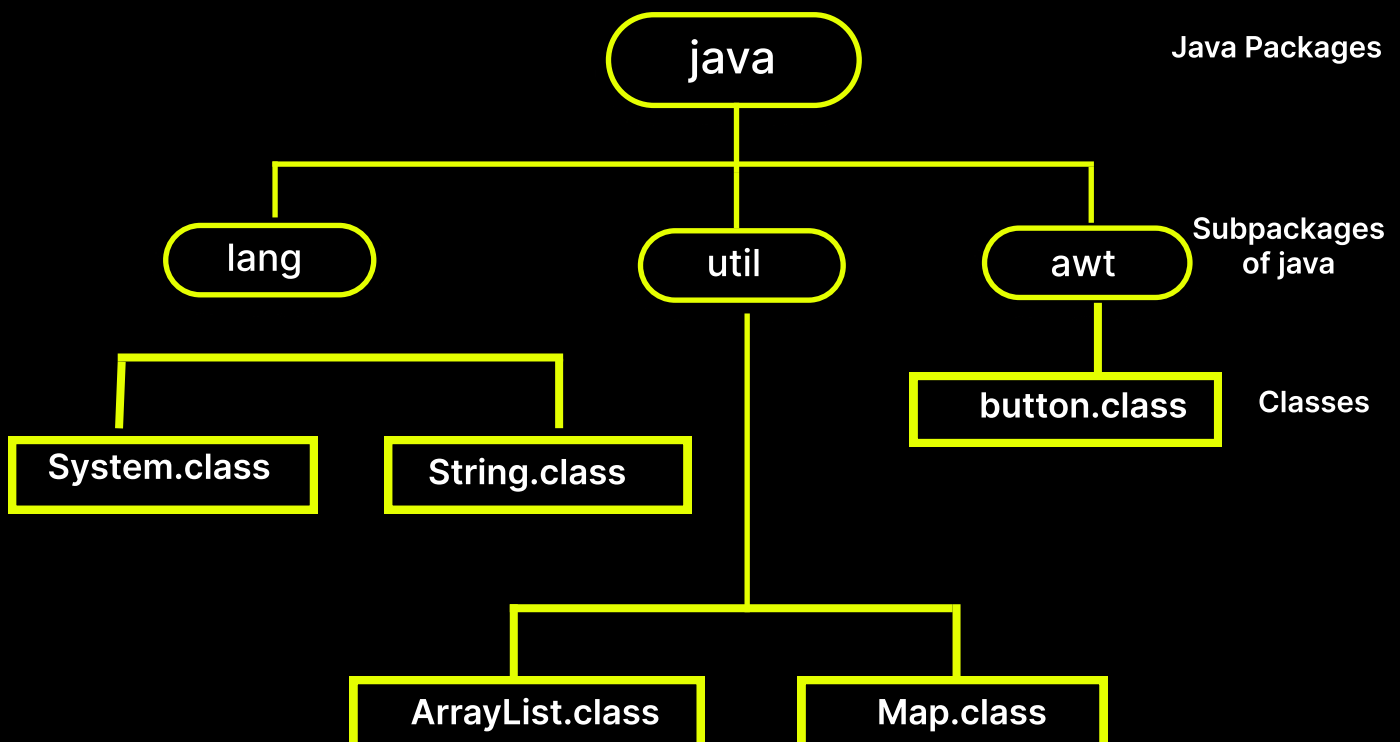
A Java package is a collection of similar types of sub-packages, interfaces, and classes.

They help you manage and group related classes, interfaces, and sub-packages to avoid naming conflicts and create a more organised and maintainable codebase.

Example:

Directories or folders on your computer's file system(manage files). In Java, there are two types of packages: built-in packages and user-defined packages.

Built-in Packages : They are available in Java, including util, lang, awt etc. We can import all members of a package using package name.* statement



java.lang is a special package that is automatically **imported by default** in every Java class.

Commonly used classes and types from the java.lang package include: String, System, Math etc.

User-defined packages: User-defined packages are those that the users define. Inside a package, you can have Java files like classes, interfaces, and a package as well (called a sub-package).

Math Class

java.lang.Math class is a built-in class. It provides mathematical functions and constants for mathematical operations.

Commonly used methods and constants:

Math.abs(a)	Returns the absolute value of a value.
Math.sqrt(a)	Returns the sqrt root of a double value.
Math.ceil(a)	Returns the closest value that is $\geq$ to the argument.
Math.floor(a)	Returns the closest value that is $\leq$ to the argument.
Math.max(a,b)	Returns the greater of two values
Math.min(a,b)	Returns the smaller of two values
Math.pow(a,b)	Returns a raised to the power b
Math.random()	Returns a double value with a +ve sign ≥ 0.0 and < 1.0

Day 8

CONTROL-FLOW STATEMENTS

Control Flow statements in programming control the order of execution of statements within a program. They allow you to make decisions, repeat actions, and control the flow of your code based on conditions.

Types of control flow statements

1. Conditional or Decision Making statements (if-else and switch)
2. Looping statements (for, while, and do-while)
3. Branching statements (break and continue)

1. Conditional statements If-else :

The if-else statement allows you to execute a block of code conditionally. If the condition inside the if statement is true, the code inside the if block is executed; otherwise, the code inside the else block is executed.

Syntax of if-else :

If-Else-If Ladder :

- "If-Else-If" ladder consists of an if statement followed by multiple else-if statements.
- It is used to evaluate a condition using multiple statements. The chain of if statements are executed from the top-down.
- It checks each if condition, and as soon as one of the if condition yields true, it executes the statement inside that if block and skip the rest of the ladder. If none of the conditions evaluates to be true, then the program executes the statement of the final else block.



```
int number = 10;
if (number % 2 == 0) {
    System.out.println("Number is even.");
}
else if (number % 2 != 0) {
    System.out.println("Number is odd.");
}
else {
    System.out.println("Invalid input.");
}
```

Output :

Number is even.

If Ladder :

- "If" ladder consists of an multiple if statements.
- It is used to evaluate a condition using multiple statements. The chain of if statements are executed from the top-down.
- The program checks each if condition, and as soon as one of the if condition yields true, it executes the statement inside that if block and still check further conditions. If none of the conditions evaluates to be true, then the program executes the statement of the final else block.

```
int number = 10;
if (number > 0) {
    System.out.println("Number is positive.");
}
if (number < 20) {
    System.out.println("Number is less than 20.");
}
if (number % 2 == 0) {
    System.out.println("Number is even.");
}
```

Output :

Number is positive.

Number is less than 20.

Number is even.

Day 9

Ternary Operator

The ternary operator, also known as the conditional operator, is a shorthand way of writing an if-else statement with a single expression.

- If the **condition** is true, the expression before the **:** (i.e., **expression1**) is evaluated and returned.
- If the **condition** is false, the expression after the **:** (i.e., **expression2**) is evaluated and returned



```
int num = ;  
String result = (num % 2 == 0) ? "Even" : "Odd";  
System.out.println("The number is " + result);
```

Type Conversion

Type casting in Java is the process of converting one data type to another. It can be done automatically or manually.

Type Casting in Java is mainly of two types.

1. Widening or Implicit Type Casting
2. Narrow or Explicit Type Casting

1. Widening or Implicit Conversion:

- Java allows automatic type conversion when a smaller data type is promoted to a larger data type.
- It is secure since there is no possibility of data loss.
- Both the data types must be compatible with each other :
converting a string to an integer is not possible as the string may contain alphabets that cannot be converted to digits.

Order : byte->short->int->long->float->double
char->int



```
int intValue = 42;  
double doubleValue = intValue; // Implicit conversion
```

2. Explicit or Narrowing Conversion:

- Sometimes, we need to convert a larger data type to a smaller one explicitly and it requires a cast operator.
- Narrowing Type Casting in Java is not secure as loss of data can occur due to a shorter range of supported values in lower data type.



```
double doubleValue = 42.0;  
int intValue = (int) doubleValue; // Explicit conversion (casting)
```

Note: Shorthand operators do implicit conversion.

Byte b = 1;

b=b+2; // error , 2 is int(all non-float by default int) so can't store in byte

b += 2; // works perfectly as += did implicit conversion

Day 10

Loops

When we want to perform certain tasks again and again till a given condition.

For e.g. : Our daily routine, certain song listen again & again

Looping is a feature that facilitates the execution of a set of instructions repeatedly until a certain condition holds false.

e.g. : print 1 to 10,000 number

Types of Loop

Categorised into two main types

1. Entry Controlled

Check the loop condition before entering the loop body. If the condition is false initially, the loop body will not execute at all.

for and while loops are examples of entry-controlled loops as we check the condition first and then evaluate the body of the loop..

a. for loop

When we know the exact number of times the loop is going to run, we use for loop.

Syntax :



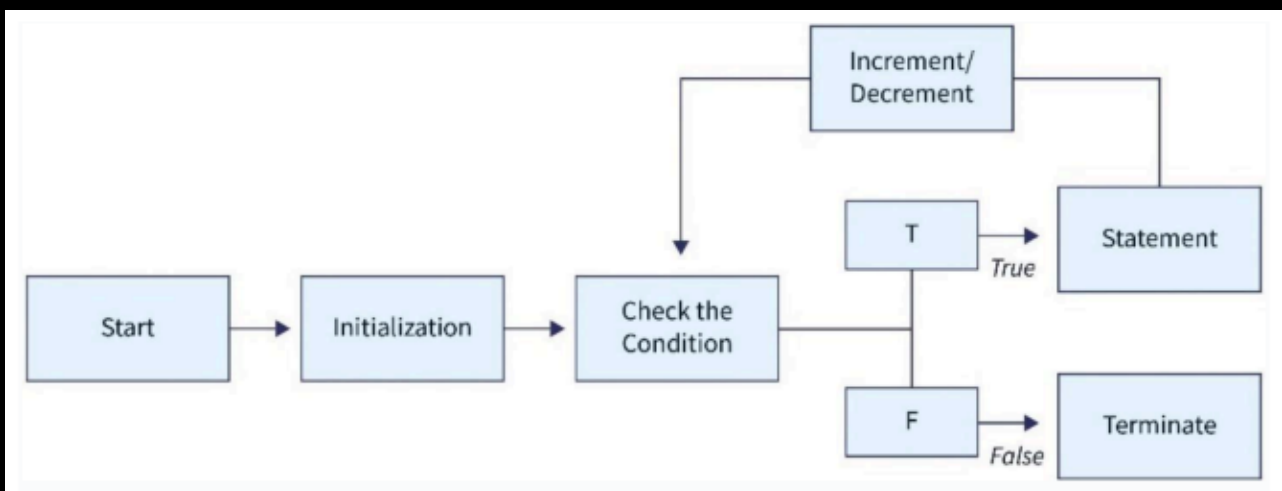
```
for(Start ; Condition ; Change){  
    // Body of the Loop (Statement(s))  
}
```

Example :



```
for (int i = 1; i<= 5; i++) {  
    System.out.println(i); //run 1 to 5  
}
```

Flow Diagram :



Optional Expressions :

In loops, **initialization**, **condition**, & **change** all are optional. Any or all of these are skippable. The loop essentially works based on the semicolon ;

```
// Infinite loop
for (;;) {}
for(;;);
for (int i = 0;; i++) {}
```

Syntax Tweaks :

- Initialize the variable outside the loop.
- Multiple conditions.
- Increment or Decrement of variable inside loop body

An infinite loop is a loop that continues executing indefinitely, and it doesn't have a condition that will terminate the loop naturally.

```
for (;;) {
    System.out.println("This is an infinite loop");
}
```

In the above code there is no initialization, no condition, and no iteration expression, meaning it will run indefinitely unless explicitly terminated.

Day 11

while loop

The while loop is used when the number of iterations is not known but the terminating condition is known.

Loop is executed until the given condition evaluates to false.

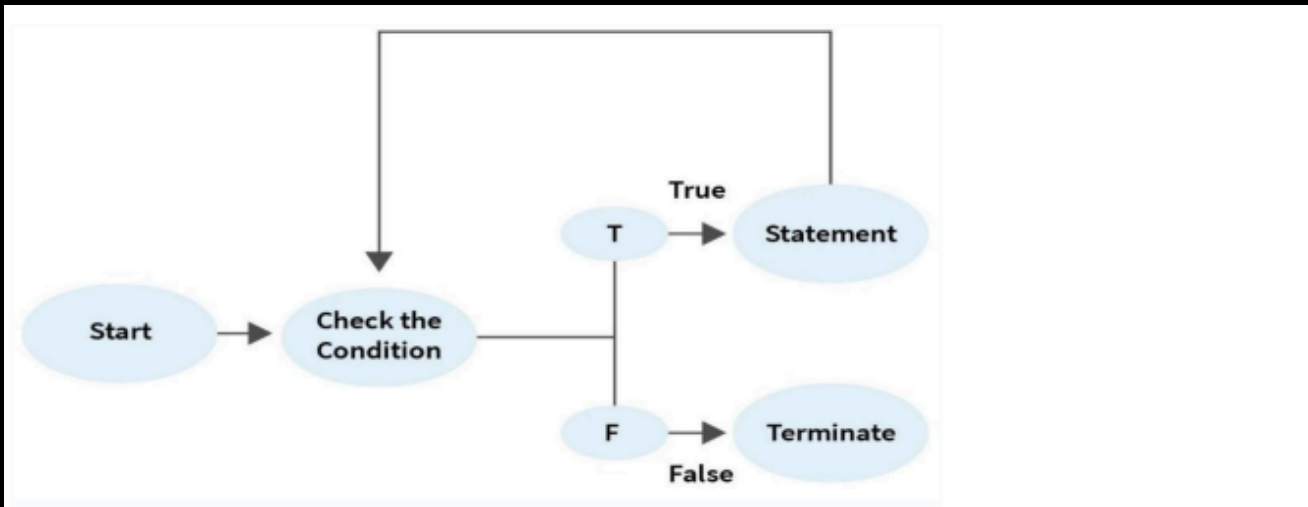
Syntax :

```
//start
while (condition){
    //Body of the loop
    //change
}
```

Example :

```
int i=0;
while (i<5){
    System.out.println(i);
    i++;
}
```

Flow Diagram :



While always accepts true, if you initially give a false condition (not Boolean false) it will neither give a syntax error nor enter in the loop.



```
int i=0;
while (i>9){// false condition but no syntax error
    System.out.println(i);
}
```

While loop always accepts true ,if you initially give false(Boolean value) it will give syntax error.



```
while (false){ //Syntax error
    System.out.println("Hello LOLU");
}
```

Day 12

do-while Loop

The do-while loop is like the while loop except that the condition is checked after evaluation of the body of the loop. Thus, the do-while loop is an example of an **exit-controlled loop**.

This loop runs at least once irrespective of the test condition, and at most as many times the test condition evaluates to true.

Syntax :

```

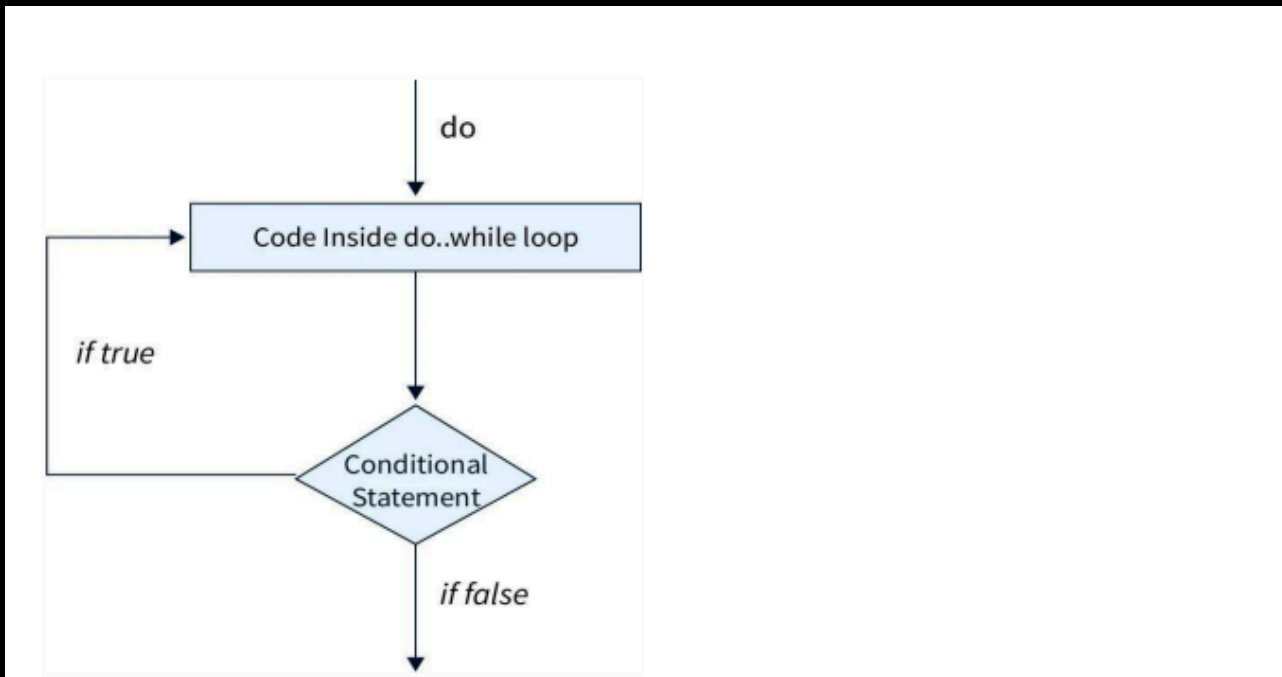
//start;
do {
    // Body of the loop (Statement(s))
    // change
}
while(condition);
```

Example :

```

int i=1;
do {
    System.out.println("Hii");
    i++;
}while (i<3);
```

Flow Diagram :



The code inside the do while loop will be executed in the first step. Then after updating the loop variable, we will check the necessary condition; if the condition satisfies, the code inside the do while loop will be executed again. This will continue until the provided condition is not true.

Infinitive do-while Loop :

```
int i = 0;
do{
    i++;
}while(i > -1);
```

There will be no output for the above code also, the code will never end. Value of initialize to 0 then increment by 1 so it can never be -1 hence the loop will never end.

Day 13

Switch Statements

The switch statement is a control flow statement that allows you to select one of many code blocks to be executed based on the value of an expression. In simple words, the Java switch statement executes one statement from multiple conditions.

```
switch(expression) {  
  case x:  
    // code block  
    break;  
  case y:  
    // code block  
    break;  
  default: // optional  
            // code block to be executed if no cases match  
            // break  
}
```

Example :

```
char ch = 'a';  
switch (ch) {  
  case 'a':  
    System.out.println("Vowel");  
    break;  
  case 'e':  
    System.out.println("Vowel");  
    break;  
  case 'i':  
    System.out.println("Vowel");  
    break;  
}
```



```
case 'o':  
    System.out.println("Vowel");  
    break;  
case 'u':  
    System.out.println("Vowel");  
    break;  
default:  
    System.out.println("Consonant");  
}
```

Important Points about Java's switch statement:

- **No variables:** The case value must be a literal or constant.
- **No duplicates:** No two cases should be of same value. Otherwise, a compilation error is thrown.
- **Allowed Types:** int, long, byte, short and String type. Primitives are allowed with their wrapper types.
- **Optional Break Statement:** Break statement is optional. If a case is matched and there is no break statement mentioned, subsequent cases are executed until a break statement or end of the switch statement is encountered (**fall through condition**).
- **Optional default case:** default case value is optional. The default statement is meant to execute when there is no match between the values of the variable and the cases. **It can be placed anywhere in the switch block .**

Multiple cases can be combined together with commas

```
char ch = 'a';
switch (ch) {
case 'a','e','i','o','u' :
    System.out.println("Vowel");
    break;
default:
    System.out.println("Consonant");
}
```

Fall through statement

A fall-through statement occurs when there is no **break** statement at the end of a **case** block. When a **case** block does not have a break statement, the code execution continues to the next **case** block, even if the condition for that **case** is not met. This behavior is known as fall-through.

Example :

```
int number = 2;

switch (number) {
case 1:
    System.out.println("One");
case 2:
    System.out.println("Two");
case 3:
    System.out.println("Three");
default:
    System.out.println("Default");
}
```

It executed the code for case 2, then continued to case 3, and finally to the default block.

Arrow Switch

```
int number = 2;
switch (number) {
    case 1 -> System.out.println("One");
    case 2 -> System.out.println("Two");
    case 3 -> System.out.println("Three");
    default -> System.out.println("Default");
}
```

It simplifies code and eliminates the need for explicit break statements.

yield Keyword :

yield keyword is used in combination with the new switch expression introduced in Java 12 to return a value from a switch expression. It allows you to specify the value to be returned from a particular case block in the switch expression.

```
int dayOfWeek = 3;
String dayName = switch (dayOfWeek) {
    case 1 : yield "Monday";
    case 2 : yield "Tuesday";
    case 3 : yield "Wednesday";
    case 4 : yield "Thursday";
    case 5 : yield "Friday";
    default : yield "Unknown";
};
System.out.println("Day of the week is: " + dayName);
```

Output :Day of the week is: Wednesday

Day 14

Nested Loops

Nested loop means a loop statement inside another loop statement. That is why nested loops are also called “**loop inside loop**”.

for loops, **while** loops, and **do-while** loops, and you can nest any of these loop types inside one another.



```
for ( initialization; condition; increment ) {  
    for ( initialization; condition; increment ) {  
        // statement of inside loop  
    }  
    // statement of outer loop  
}
```

Note: There is no rule that a loop must be nested inside its own type. In fact, there can be any type of loop nested inside any type and to any level.